Correção · Algoritmos · Big-O · 12 questões

Data: 06/11/2026 · **Semana 3 (adiantada)** · **Placar: 7/12 (58%)** · 3 erros · 2 "não sabia" ("?")

Respostas lidas das anotações do PDF: 1-B · 2-C · 3-B · 4-C · 5-B · 6-"?" · 7-E · 8-D · 9-E · 10-E · 11-A · 12-"?"

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> **Como li as marcas:** usei os círculos nas letras (a tabela "Minhas respostas" ficou vazia). **Pontinho = candidata/dúvida:** Q1, Q2, Q3, Q7, Q10 e Q11 (na Q4, pontinho na A e na C). **Q5:** a marca ao lado da (B) é a ponta da seta que sai de "recursiva", então contei como acerto firme. **Q6 e Q12:** só "?", sem círculo = **não sabia**. **Q10:** a anotação "fibbonati ?" mostra que você não reconheceu o padrão. Nas Q1, Q2, Q7 e Q8 você escreveu a complexidade de cada laço ao lado do código: é exatamente o método.

Questão	Assunto	Sua	Gabarito	Resultado
1	Laços em sequência + limite fixo	B	D	❌
2	Laço que multiplica × laço linear (aninhados)	C	C	✅ (com dúvida)
3	Tempo estimado com $\Theta(n^2)$	B	B	✅ (com dúvida)
4	Busca binária × hash (restrição do cenário)	C	A	❌
5	Recursão com laço (n + (n−1) + ...)	B	B	✅
6	Notação O × Ω (I/II/III)	?	C	⬜ não sabia
7	Duplicados: laços × conjunto hash	E	E	✅ (com dúvida)
8	Duas entradas (m · n)	D	D	✅
9	Melhor × pior caso (busca linear)	E	E	✅
10	Dois ponteiros	E	B	❌
11	Constante × classe ($100n \times n^2$)	A	A	✅ (com dúvida)
12	Recursão que divide ao meio (merge sort)	?	D	⬜ não sabia

Por tipo:

Tipo	Questões	Placar
Laços (contar voltas)	1, 2, 8, 10	2/4
Recursão	5, 12	1/2 (Q12 "?")
Casos e comparação (melhor × pior, constante × classe, tempo estimado)	3, 9, 11	3/3
Escolha de estrutura e notação	4, 6, 7	1/3 (Q6 "?")

Acertos firmes: 3 (Q5, Q8, Q9) · **com dúvida:** 4 (Q2, Q3, Q7, Q11).

Comparação: questões do chat (06/11, à tarde) 3 acertos em 5 respondidas, um deles pelo motivo errado → **bloco 7/12**. O erro do chat (inverter soma × multiplicação) **não se repetiu**: na Q2 você escreveu "log n" para `i = i * 2` e "O(n)" para o laço de dentro. O erro novo é outro: na Q1 você **multiplicou dois laços que estão em sequência**.

Questão 1 · ❌ Errou

O analista Fábio revisa a função a seguir, do sistema e-CUSTAS, que consolida os valores de uma lista com n processos e, para cada processo, os valores dos 12 meses do ano:

```
def consolida(processos):
    n = len(processos)
    total = 0
    for p in processos:
        total += p.valor
    for i in range(n):
        for mes in range(12):
            total += valor_mensal(processos[i], mes)   # custo constante
    return total
```

A complexidade de tempo da função consolida, em função de n, é:

(A) $O(2^n)$;

(B) $O(n^2)$; ← sua resposta

(C) O(n log n);

✅ **(D) O(n);**

(E) O(log n).

Onde você se confundiu: a análise que você escreveu na margem estava **certa**: "O(n)" no primeiro laço e "O(n)" no segundo bloco (você percebeu que o `range(12)` não depende de n). Mas os dois blocos estão **um depois do outro**, não um dentro do outro. Em sequência, **soma**: n + 12n = 13n → O(n). Você multiplicou O(n) × O(n) e marcou $O(n^2)$.

📌 **Blocos em sequência (mesma indentação, um depois do outro) SOMAM: O(n) + O(n) = O(n). Só multiplica o que está DENTRO de outro laço.** Na margem, escreva "+" entre os blocos de fora e "×" só entre laço e laço interno.

Questão 2 · ✅ Acertou (com dúvida)

A analista Patrícia encontrou a rotina a seguir no sistema SIGEP-PB, escrita em Java, que reprocessa um lote com n registros:

```
for (int i = 1; i < n; i = i * 2) {
    for (int j = 0; j < n; j++) {
        lote[j] = lote[j] + i;
```

```
    }
}
```

A complexidade de tempo da rotina é:

(A) O(log n);

(B) O(n);

✅ **(C) O(n log n);** ← sua resposta

(D) $O(n^2)$;

(E) $O(n^2 \log n)$.

Você escreveu "log n" ao lado de `i = i * 2` e "O(n)" no laço interno: aninhados multiplicam → n log n. O erro do chat (soma × multiplicação invertidas) foi corrigido aqui.

📌 Laço de fora que **multiplica** (log n) × laço de dentro linear (n) = **O(n log n)**. A ordem dos laços não importa: log n × n = n × log n.

Questão 3 · ✅ Acertou (com dúvida)

A rotina de geração do relatório de produtividade do sistema e-METAS tem tempo de execução $\Theta(n^2)$, em que n é o número de processos analisados. Em testes, a rotina levou 4 segundos para processar 10.000 processos.

Mantidas as demais condições, o tempo estimado para processar 30.000 processos é:

(A) 108 segundos;

✅ **(B) 36 segundos;** ← sua resposta

(C) 24 segundos;

(D) 16 segundos;

(E) 12 segundos.

Sua conta na margem estava perfeita: 100 → 4, 900 → x, x = 36 (você elevou 10 e 30 ao quadrado). 12 s seria linear e 108 s, cúbico.

📌 **Tempo estimado: a entrada cresce k vezes → O(n) multiplica o tempo por k · $O(n^2)$ por k^2 · $O(n^3)$ por k^3.** 3× mais dados em $O(n^2)$ = 9× o tempo.

Questão 4 · ❌ Errou

O sistema e-CONSULTA mantém em memória um vetor com 2 milhões de números de processos, ordenado e sem alterações durante o expediente. O sistema recebe milhares de consultas por segundo para verificar se um número existe no vetor. A equipe do analista Henrique precisa da estratégia com o menor tempo de pior caso, sem usar memória adicional.

A estratégia mais adequada e a sua complexidade de pior caso são:

✅ **(A) busca binária, com complexidade O(log n);**

(B) busca linear, com complexidade O(n), já que o vetor ordenado permite parar ao encontrar o número;

(C) tabela hash montada a partir do vetor, com complexidade O(1) no pior caso; ← sua resposta

(D) busca binária, com complexidade O(n log n), pois o vetor precisa ser reordenado a cada consulta;

(E) busca linear, com complexidade O(log n), pois o vetor está ordenado.

Onde você se confundiu: você riscou certo a B e a E e ficou entre A e C (pontinho nas duas). Duas pistas do cenário eliminavam a C. **"Sem usar memória adicional":** montar uma tabela hash cria uma estrutura nova, do tamanho do vetor. **"Pior caso":** o O(1) da hash é o **caso médio**. No pior caso, com muitas colisões, a busca na hash vira **O(n)**. A busca binária usa o próprio vetor ordenado e garante O(log n) mesmo no pior caso.

📌 **Hash = O(1) no caso MÉDIO e O(n) no PIOR caso (colisões), e gasta memória extra. Vetor ordenado + "pior caso" + "sem memória adicional" = busca binária, O(log n).** No modelo Tribunal, o cenário traz uma restrição decisiva, e cada distrator viola essa restrição. Sublinhe a restrição antes de ler as alternativas.

Questão 5 · ✅ Acertou

A analista Simone encontrou, no módulo de cálculos do sistema e-DEPÓSITO, a função recursiva a seguir, chamada inicialmente com n igual ao tamanho do vetor v:

```
def soma_prefixos(v, n):
    if n == 0:
        return 0
    parcial = 0
    for i in range(n):
        parcial += v[i]
    return parcial + soma_prefixos(v, n - 1)
```

A complexidade de tempo da função soma_prefixos é:

(A) $O(2^n)$;

✅ **(B) $O(n^2)$;** ← sua resposta

(C) O(n log n);

(D) O(n);

(E) O(log n).

Uma chamada por nível (n, n−1, n−2, ...) e, em cada nível, um laço com o tamanho atual: n + (n−1) + ... + 1 ≈ $n^2/2$ → $O(n^2)$. É o "triângulo" em forma de recursão. Não é $O(2^n)$ porque cada chamada abre só **uma** outra.

📌 **Recursão com 1 chamada (n − 1) e um laço de n dentro = triângulo = $O(n^2)$.** Sem o laço, seria O(n), como o fatorial.

Questão 6 · ⬜ Não sabia ("?")

Em um treinamento sobre análise de algoritmos para a equipe de desenvolvimento de um Tribunal de Justiça, foram apresentadas as afirmativas a seguir.

I. Se o número de operações de um algoritmo é $f(n) = 3n^2 + 50n + 1000$, então $f(n)$ é $O(n^2)$.

II. Se um algoritmo é $O(n^2)$, então ele também é $O(n^3)$.

III. A notação Ω (ômega) descreve um limite superior assintótico para o tempo de execução de um algoritmo.

Está correto o que se afirma em

(A) I, apenas.

(B) III, apenas.

✅ **(C) I e II, apenas.**

(D) II e III, apenas.

(E) I, II e III.

O que faltou (pelas suas marcas):

- **I** ("bascara?"): não precisa resolver nada. É a regra do termo dominante: em $3n^2 + 50n + 1000$, fica o n^2 e a constante 3 cai → $O(n^2)$. **Verdadeira.**
- **II** (você marcou X): parece estranha, mas é **verdadeira**. O Big-O é um **teto** ("no máximo"). Se o algoritmo cresce no máximo como n^2, também cresce no máximo como n^3, que é um teto mais alto. O teto não precisa ser justo; quando é justo, usa-se Θ.
- **III** ("?"): **falsa**. Ω é o **piso** (limite inferior, "no mínimo"). O teto é o O.

Mesmo sem saber a III, I = V já eliminava B e D. Com I = V e III = F, sobravam A e C, e era só decidir a II.

📌 **O = teto ("no máximo") · Ω = piso ("no mínimo") · Θ = os dois ao mesmo tempo (crescimento exato).** Se f é $O(n^2)$, também é $O(n^3)$, $O(2^n)$ etc., porque todo teto mais alto vale. Em polinômio, fica o maior expoente e as constantes caem.

Questão 7 · ✅ Acertou (com dúvida)

Para impedir o cadastro de guias com número repetido, o analista Diego escreveu duas versões de uma função para o sistema e-GUIAS, que recebe uma lista com n números:

```
def tem_duplicado_v1(numeros):
    for i in range(len(numeros)):
        for j in range(i + 1, len(numeros)):
            if numeros[i] == numeros[j]:
                return True
    return False
```

```
def tem_duplicado_v2(numeros):
    vistos = set()            # conjunto implementado com tabela hash
    for x in numeros:
        if x in vistos:
            return True
        vistos.add(x)
    return False
```

Considerando o pior caso das funções e o custo médio das operações sobre o conjunto, as complexidades de tempo de tem_duplicado_v1 e tem_duplicado_v2 são, respectivamente:

(A) O(n) e $O(n^2)$;

(B) $O(n^2)$ e $O(n^2)$, pois as duas versões percorrem a lista inteira;

(C) O(n log n) e O(n);

(D) $O(n^2)$ e O(1);

✅ **(E) $O(n^2)$ e O(n).** ← sua resposta

v1 é o triângulo (j começa em i + 1) → $O(n^2)$. v2 faz um laço de n e, em cada volta, consulta e insere no conjunto a O(1) médio → O(n). (D) erra porque o laço da v2 ainda percorre os n itens.

📌 **Trocar a busca numa lista (O(n)) por consulta em set/dict (O(1) médio) tira um nível de laço: $O(n^2)$ → O(n).** É a otimização mais cobrada em leitura de código.

Questão 8 · ✅ Acertou

O sistema e-DISTRIBUI, de um Tribunal de Justiça, conta quantos processos pertencem a cada vara. O trecho em Java a seguir recebe uma lista com m varas e uma lista com n processos:

```
for (Vara v : varas) {
    for (Processo p : processos) {
        if (p.getVaraId() == v.getId()) {
            contagem[v.getId()]++;
        }
    }
}
for (Vara v : varas) {
    imprime(v, contagem[v.getId()]);
}
```

A complexidade de tempo do trecho é:

(A) O(n);

(B) O(m + n);

(C) $O(n^2)$;

✅ **(D) O(m · n);** ← sua resposta

(E) O(m).

Laço de m com laço de n dentro = m · n. O segundo laço (m), **em sequência**, soma: m · n + m → fica O(m · n). Aqui você somou o bloco de baixo corretamente.

📌 **Entradas diferentes ganham letras diferentes: m varas × n processos = O(m · n), não O(n^2).**

Questão 9 · ✅ Acertou

A analista Lúcia analisa o método a seguir, do sistema e-PROTOCOLO, que procura um número de protocolo num vetor não ordenado com n posições:

```
public int localiza(String[] numeros, String alvo) {
    for (int i = 0; i < numeros.length; i++) {
        if (numeros[i].equals(alvo)) {
            return i;
        }
    }
    return -1;
}
```

As complexidades de tempo do método no melhor caso e no pior caso são, respectivamente:

(A) O(n) e O(n);

(B) O(1) e O(1);

(C) O(log n) e O(n);

(D) O(n) e O(1);

✅ **(E) O(1) e O(n).** ← sua resposta

Você riscou B, C e D com segurança. Melhor caso: o número está na 1ª posição (1 volta). Pior caso: não existe ou está no fim (n voltas). (D) inverte a ordem do "respectivamente".

📌 **Busca linear: melhor caso O(1), pior caso O(n). Ordenar o vetor NÃO melhora a busca linear (TJ-RN 2023); só a busca binária aproveita a ordenação.**

Questão 10 · ❌ Errou

O sistema e-QUITAÇÃO verifica se existem duas guias, num vetor de n valores ordenado em ordem crescente, cuja soma é exatamente o valor devido. O analista Marcelo implementou a função a seguir:

```
def existe_par(v, alvo):
    i, j = 0, len(v) - 1
    while i < j:
        soma = v[i] + v[j]
        if soma == alvo:
```

```
            return True
        elif soma < alvo:
            i += 1
        else:
            j -= 1
    return False
```

A complexidade de tempo da função, no pior caso, é:

(A) O(log n);

✅ **(B) O(n);**

(C) O(n log n);

(D) O(n^2);

(E) O(2^n). ← sua resposta

Onde você se confundiu: você anotou "fibbonati ?": as **duas variáveis** e a **soma** `v[i] + v[j]` lembraram o Fibonacci (fib(n−1) + fib(n−2)). Mas aqui não há recursão: é **um único `while`**. Use a técnica: o que acontece com as variáveis a cada volta? `i += 1` (**soma**) ou `j -= 1` (**subtrai**), e só uma das duas por volta. Elas começam nas pontas e vão se aproximando. Com n = 8, no máximo 7 voltas até `i` encontrar `j`. São **n voltas no máximo → O(n)**. Esse padrão se chama **dois ponteiros**.

📌 **Dois ponteiros (i no começo, j no fim, um anda por volta até se encontrarem) = O(n).** Exponencial só aparece quando uma função **chama a si mesma duas vezes**. Soma de valores dentro do laço não é recursão.

Questão 11 · ✅ Acertou (com dúvida)

Dois fornecedores apresentaram ao Tribunal algoritmos para ordenar a fila de distribuição de processos. Para uma entrada com n processos, o algoritmo X executa 100n operações e o algoritmo Y executa n^2 operações.

Sobre os dois algoritmos, é correto afirmar que:

✅ **(A) para entradas com menos de 100 processos, Y executa menos operações, mas, acima de 100, X passa a ser mais eficiente, pois X é O(n) e Y é O(n^2);** ← sua resposta

(B) X é mais eficiente para qualquer tamanho de entrada, pois tem complexidade linear;

(C) Y é mais eficiente para qualquer tamanho de entrada, pois não tem constante multiplicativa;

(D) X e Y têm a mesma complexidade assintótica, já que executam o mesmo número de operações quando n = 100;

(E) X é O(100n) e, por isso, é menos eficiente do que Y para entradas grandes.

Teste com n = 10: X = 1.000 e Y = 100 (Y ganha). Com n = 1.000: X = 100.000 e Y = 1.000.000 (X ganha). Empatam em n = 100. Big-O descreve o comportamento para **n grande**; para entradas pequenas, a constante pode inverter o resultado.

📌 **Constante grande pode fazer o algoritmo O(n) perder para o $O(n^2)$ em entradas pequenas, mas, a partir de certo n, a classe menor sempre vence. "O(100n)" é O(n).**

Questão 12 · ⬜ Não sabia ("?")

O analista Anderson encontrou, no sistema e-ARQUIVO, a função recursiva a seguir, que organiza um vetor v com n documentos. A função junta percorre uma única vez os elementos de esq e dir, com custo proporcional ao tamanho dos dois juntos:

```
def organiza(v):
    if len(v) <= 1:
        return v
    meio = len(v) // 2
    esq = organiza(v[:meio])
    dir = organiza(v[meio:])
    return junta(esq, dir)
```

A complexidade de tempo da função organiza é:

(A) O(log n);

(B) O(√n);

(C) O(n);

✅ **(D) O(n log n);**

(E) $O(n^2)$.

É o **merge sort**. Desenhe a árvore:

```
nível 0:            [8]                 → junta 8 itens
nível 1:        [4]     [4]             → junta 4 + 4 = 8
nível 2:      [2] [2] [2] [2]           → 2+2+2+2 = 8
nível 3:   [1][1][1][1][1][1][1][1]     → fim
```

Cada nível divide ao meio → **log n níveis** (8 → 4 → 2 → 1 = 3 níveis). Em **cada nível**, o `junta` passa por **n itens no total**. Total: n por nível × log n níveis = **O(n log n)**. Não é $O(2^n)$ porque, embora abra 2 chamadas, cada uma recebe **metade** (não n − 1).

📌 **2 chamadas com n/2 + trabalho de n em cada nível = O(n log n) (merge sort). Compare: 2 chamadas com n − 1 = $O(2^n)$ · 1 chamada com n/2 = O(log n).**

Para o caderno de erros

Assunto	Por que errei	Regra certa
Laços em sequência (Q1)	Analisei certo (O(n) + O(n)), mas multipliquei	Sequência soma; só o que está dentro multiplica
Hash × busca binária (Q4)	Aceitei "O(1) no pior caso" e ignorei "sem memória adicional"	Hash: O(1) médio, O(n) pior, gasta memória. Vetor ordenado → binária O(log n)
O × Ω (Q6)	Não sabia Ω; achei que $O(n^2)$ não pode ser $O(n^3)$	O = teto, Ω = piso, Θ = exato. Teto mais alto também vale
Dois ponteiros (Q10)	Duas variáveis + soma lembraram Fibonacci	Um while, cada volta anda 1 → O(n). Exponencial = função que chama a si mesma 2× com n − 1
Merge sort (Q12)	Recursão que divide em 2	log n níveis × n por nível = O(n log n)

O que estudar agora

Quando	O que fazer
Agora, 10 min	Refazer **Q1, Q10 e Q12 sem olhar**, escrevendo na margem "+" ou "×" entre os blocos e desenhando a árvore da Q12 com n = 8
22:40 · revisão	Regras 📌 de hoje (A3 e Big-O) · Anki · refazer Q8, Q13 e Q15 do A3
Sáb 07/11	Bloco de **APF funções de dados** (já pronto) + responder a **Q5 do chat** (`range(5)`)
Dom 08/11 · 13h	1º simulado oficial
Próxima noite de Específicas	**Listas, pilhas, filas, árvores e hash** (fila nº 11). A Q4 (hash × busca binária) já adianta parte disso